

End-to-End Automation Blueprint

From 15 manual touchpoints to a single launch trigger

A complete read of the repository — 153 REST endpoints, 35 database tables, 45 React pages, 4 email dispatchers and 5 scheduler modules — and a concrete plan to collapse the entire batch lifecycle into one trigger, with every downstream email, push notification, reminder, window and escalation derived automatically.

WHAT THIS DOCUMENT ANSWERS

- Can one trigger really run the whole batch? Yes - the last mile already exists.
- What is actually blocking it today? Ten specific, named defects (Section 4).
- Deterministic orchestrator or agentic AI? Both, on different halves of the problem.
- How long? 4-6 weeks, phased, with each phase shippable on its own.

REPOSITORY	engg-automation-git-file
BRANCH ANALYSED	rollback-stable (HEAD 29eab18)
SCOPE	Full read-only analysis. No application code was modified.
PREPARED	04 August 2026

Contents

1. Executive Summary	3
The single most important finding	3
The gap	3
Recommendation in one line	3
Headline numbers	3
2. What Exists Today - System Inventory	5
2.1 Stack	5
2.2 The domain model	5
2.3 The delivery queue (the part that already works)	5
2.4 Notification fan-out	6
3. The Batch Lifecycle As It Runs Today	7
3.1 The most wasteful gap: steps 4 to 6	7
3.2 Dates are re-typed, not derived	7
4. What Blocks Full Automation - Ten Named Defects	9
B1 Four competing email dispatchers	9
B2 The automation is written but never invoked	9
B3 Critical state stored on an ephemeral filesystem	9
B4 marks_entry_windows is never populated	10
B5 Authentication is absent, and the token is forgeable	10
B6 Live credentials committed to the repository	10
B7 No schema as code, no CI, no tests	10
B8 A second abandoned application	10
B9 No workflow state model	11
B10 Fragile string matching and duplicated business rules	11
5. Target Architecture - The One-Point Trigger	12
5.1 The trigger	12
5.2 New tables	12
5.3 The stage graph	13
5.4 The reconciler	14
5.5 Idempotency	15
5.6 Unified dispatch	16
6. Option A - Deterministic Orchestrator (the spine)	17
6.1 What it automates completely	17
6.2 Rules become data, not code	17
6.3 The operator console	17
6.4 Handling reschedules	18
7. Option B - The Claude Code Agentic Layer	19
7.1 Where an agent must NOT be used	19
7.2 Five agents that do earn their place	19
7.3 How the runtime agents are hosted	20
7.4 Guardrails	20
8. Recommendation and Roadmap	22
8.1 Recommended shape - hybrid, deterministic spine first	22
8.2 Phased plan	22
8.3 Effort and sequencing	23
8.4 What a human still does afterwards	23
8.5 Risks	23
9. Appendix - Evidence Index	24
Scope note	24

1. Executive Summary

The platform is much closer to full automation than it looks. The hard part - a durable, retrying, deduplicated delivery queue - is already built and running in production. What is missing is the layer above it: nothing decides what should be scheduled. A human does, by opening a page, picking a batch and a date, and pressing a button. Fifteen times per batch.

The single most important finding

STRENGTH THE DELIVERY ENGINE ALREADY EXISTS AND WORKS

backend/index.js:9815 runs a per-minute node-cron job over a scheduled_emails table. It has optimistic row locking, a processing state, retry_count with MAX_RETRIES = 3, terminal permanently_failed status, batch size limiting and catch-up semantics (scheduled_at <= now, so nothing is lost during downtime). That is a real job queue. Every automation described in this document ends by writing a row into it - no new delivery infrastructure is required.

Alongside it, push notification fan-out (backend/push/sendPush.js) is already role-aware, fire-and-forget, and self-healing - it prunes dead FCM tokens automatically and is wired into 22 event sites. And email_templates already carries offset_days plus send_time, which means the system *already understands* the idea of "send this N days after batch start at this time". That is the seed of a declarative rule engine.

The gap

The course planner (course_planner_data) holds every dated obligation a batch will ever have: every weekly quiz, every intermediate assessment, every final assessment, the lab sessions, the trainer assigned to each, the last teaching day. It is a complete, machine-readable schedule. Yet almost nothing reads it to drive automation. Instead a coordinator re-enters those same dates by hand into the Mock Interview page, the Soft Skill page, the Final Assessment page, the Course Closure page, and the mark window controls.

BLOCKER THE AUTOMATION WAS WRITTEN, THEN NEVER SWITCHED ON

Four scheduler modules - scheduleWeeklyQuizReminders, scheduleIntermediateAssessmentReminders, scheduleCourseApplicationEmails and scheduleInternalEmail - are imported at the top of index.js (lines 17-21) and never called anywhere in the codebase. workers/sendMarksReminders.js is exported and never imported. The logic to derive reminders from the planner largely exists; it is simply not wired to anything.

Recommendation in one line

Build a deterministic Batch Lifecycle Orchestrator - a launch endpoint, a task table, and a reconciler tick - that reads the course planner and populates the queue that already works. Then put Claude-based agents on the edges, where judgement is genuinely required: classifying messy planner rows, triaging exceptions, drafting communications and auditing data quality. Do not put an LLM anywhere on the critical path of date arithmetic, eligibility rules or marks.

Headline numbers

METRIC	TODAY	AFTER
Manual actions to run one batch end-to-end	~15	1 (launch)
Dates re-typed by a human per batch	8-12	0
Email dispatchers running concurrently	up to 4	1
Mutating endpoints requiring authentication	0 of ~110	all
Workflow state visible to operators	none	full run timeline

METRIC	TODAY	AFTER
Recovery when a step fails	manual re-click	automatic retry + escalation
Reminder schedulers actually executing	0 of 5	5 of 5

The remaining human work after automation is the work that should stay human: entering marks, marking attendance, reviewing the scorecard, and approving exceptions the system flags.

2. What Exists Today - System Inventory

2.1 Stack

LAYER	TECHNOLOGY	NOTES
Backend	Node >=18, Express 4, ESM	backend/index.js is 10,763 lines and holds 153 routes
Database	Supabase / PostgreSQL	35 tables; accessed with the service-role key (bypasses RLS)
Direct SQL	pg Pool (backend/db.js)	Used only by marksWindowService and the marks routers
Email	Mailjet v3.1 via raw fetch	backend/emailSender.js - clean, well-logged, returns {success,error}
Push	Firebase Admin (FCM)	backend/push/ - multicast, auto-prunes invalid tokens
Scheduling	node-cron	Two live jobs: per-minute dispatcher, 10-minute keep-alive ping
Documents	PDFKit, ExcelJS, xlsx	Report and planner export
Planner build	Python 3 + openpyxl (vendored)	backend/scripts/, spawned as a child process
Frontend	React 18, MUI 7, CRA	45 pages/components; role menus in App.js
Mobile	Capacitor 6	Wraps the hosted site; native push
Hosting	Render	RENDER_EXTERNAL_URL keep-alive suggests a free/sleeping instance

2.2 The domain model

Thirty-five tables are referenced from the backend. They group cleanly into six concerns, which is useful because the orchestrator's stages map almost one-to-one onto these groups.

CONCERN	TABLES
Identity	internal_users, users, device_tokens
Batch setup	batches, batch_table, domains, classrooms, classroom_occupancy, licenses, holidays
Schedule	course_planner, course_planner_data, course_module_template, date_change_audit
People	learners_data, trainer_domain, batch_module_trainers, batch_trainer_conflicts, trainer_unavailability
Assessment	weekly_assessment_scores, intermediate_assessment_scores, module_level_assessment_scores, final_assessment_scores, final_project_scores, viva_scores, scorecard_overrides, marks_entry_windows, marks_entry_extension_requests, status_unlock_requests
Delivery	scheduled_emails, email_templates, internal_email_templates, batch_forms, forms, learner_attendance

DESIGN COURSE_PLANNER_DATA IS THE SOURCE OF TRUTH

Each row carries batch_no, date, week_no, start_time, end_time, module_name, topic_name, trainer_name, trainer_email, topic_status, actual_date and batch_type. Every reminder, window, assessment mail and closure date the business needs can be derived from this one table. The automation design in Section 5 is built entirely on that observation.

2.3 The delivery queue (the part that already works)

```
// backend/index.js:9815
cron.schedule("* * * * *", async () => {
  const emails = await supabase.from("scheduled_emails")
    .select("*")
    .in("status", ["scheduled", "failed"])
    .lte("scheduled_at", nowUTC) // catch-up: nothing is lost on downtime
    .or(`retry_count.is.null,retry_count.lt.${MAX_RETRIES}`)
    .order("scheduled_at").limit(BATCH_SIZE);

  for (const email of emails) {
    // optimistic lock - only proceed if THIS tick won the row
    const { data: locked } = await supabase.from("scheduled_emails")
      .update({ status: "processing", retry_count: String(n+1) })
      .eq("id", email.id).in("status", ["scheduled", "failed"]).select("id");
    if (!locked?.length) continue; // another tick already has it

    const result = await sendRawEmail({ ... });
    status = result.success ? "sent"
      : (retry+1 >= MAX_RETRIES ? "permanently_failed" : "failed");
  }
});
```

This is the single most reusable asset in the repository. The orchestrator does not replace it - the orchestrator's only job at the end of each stage is to insert well-formed rows into this table.

2.4 Notification fan-out

backend/push/sendPush.js exposes `notify(emailsOrPromise, {title, body, data})` - a fire-and-forget helper that never throws and never blocks a response. `getRoleEmails()` and `getApproverEmails()` resolve recipients from `internal_users` by role, and invalid FCM tokens are pruned on every send. It is already called at 22 event sites (batch scheduled, planner uploaded, window extended, mark entry opened, extension approved/rejected, leave applied, learner status changed, course closing, and more).

STRENGTH PUSH IS PRODUCTION-READY AND ROLE-AWARE

This layer needs no redesign. It only needs to be invoked from the orchestrator's stage transitions instead of from ad-hoc points inside request handlers, so that a notification is a consequence of a state change rather than a side effect of someone pressing a button.

3. The Batch Lifecycle As It Runs Today

Reconstructed from the route handlers and the React pages that call them. Each numbered step is a separate human action, on a separate screen, in a required order that is documented nowhere.

#	ACTION	SCREEN	ENDPOINT	AUTOMATABLE
1	Allocate classroom + licenses	ClassroomScheduler	POST /api/scheduleBatch	Fully
2	Generate course planner xlsx	CoursePlannerGenerator	POST /api/course-planner/generate	Fully
3	Hand-edit planner grid	CoursePlannerGenerator	POST /api/course-planner/save/:id	Keep manual (review)
4	Convert planner to system CSV	CoursePlannerGenerator	POST /api/course-planner/convert	Fully
5	Download the CSV to disk	browser download	GET .../download/:id/csv	Eliminate entirely
6	Re-upload that same CSV	HomeDashboard	POST /upload-course-planner	Eliminate entirely
7	Upload learners CSV	HomeDashboard	POST /upload-learners	Fully
8	Assign trainers per module	TrainerAssignmentDashboard	POST /api/assign-batch-trainers-by-module	Assisted
9	Press Schedule Emails	HomeDashboard	POST /api/schedule-email	Fully
10	Open / extend mark windows	MarkSheet, MarkEntry	POST /api/marks/mark-entry-open	Fully
11	Approve extension requests	MarkExtensionReport	POST /api/marks/extension-requests/:id/approve	Assisted
12	Schedule mock interview	MockInterviewSchedule	POST /api/mock-interview-schedule	Fully
13	Send soft-skill announcement	SoftSkillAnnouncement	POST /api/soft-skill-announcement	Fully
14	Schedule final assessment mail	FinalAssessmentSchedule	POST /api/send-final-assessment-email	Fully
15	Trigger course closure	InternalCommunication	POST /api/course-closure	Fully

3.1 The most wasteful gap: steps 4 to 6

The planner generator writes the .xlsx and .csv to backend/uploads/course-planner/ on the server's own disk. To get that data into the database, a human downloads the CSV from the server and then uploads the identical bytes back to the same server through a different endpoint, where the browser parses it with PapaParse and POSTs JSON rows.

GAP A SERVER-TO-SERVER STEP ROUTED THROUGH A HUMAN

The file never needed to leave the machine. routes/coursePlanner.js already knows the exact CSV path (WORK_DIR/<id>.csv) and index.js already has the insert logic for course_planner_data. A single internal call collapses three manual steps to zero - and removes the class of errors where the wrong CSV, or a stale one, is uploaded to the wrong batch.

3.2 Dates are re-typed, not derived

Steps 12, 13, 14 and 15 all ask a human for a date that is already recorded in course_planner_data. The mock interview date, the soft-skills date, the final assessment dates and the course end date are all in the planner. Re-typing them is both unnecessary and a correctness risk: when a topic is rescheduled (there is a whole date_change_audit table and an /api/update-actual-date endpoint for exactly this), the already-scheduled emails

are not revised.

4. What Blocks Full Automation - Ten Named Defects

These are concrete, located, verifiable. Several must be fixed before a one-click trigger is safe to expose at all - a single endpoint that runs an entire batch is exactly the endpoint you do not want unauthenticated.

B1 Four competing email dispatchers

DISPATCHER	STATE	BEHAVIOUR
index.js:9815 cron	LIVE	Correct. Catch-up (lte now), optimistic lock, retry ceiling.
email-worker.js	npm run worker	Only picks rows inside the CURRENT minute - anything missed is never sent. Lock result is not checked, so it can double-send alongside the cron.
backend/src/scheduler.js	dead	CommonJS. Polls status = 'pending'; the live code writes 'scheduled'. Would never match.
courseApplicationScheduler.js	imported, never called	An unbounded while(true) loop. If it were ever invoked it would block the event loop.

BLOCKER DUPLICATE-SEND RISK IS REAL

If the Render worker process (npm run worker) is ever started next to the web service, both dispatchers read the same table. The worker marks rows as processing but never verifies it won the row, so learners can receive the same email twice. Choose one dispatcher - the index.js cron - and delete the rest.

B2 The automation is written but never invoked

```
// backend/index.js lines 16-22
import {
  scheduleWeeklyQuizReminders,           // <- never called
  scheduleIntermediateAssessmentReminders, // <- never called
} from "../reminderScheduler.js";
import { scheduleCourseApplicationEmails } from "../courseApplicationScheduler.js"; // never
import { scheduleInternalEmail } from "../emailScheduler.js"; // never
called
// workers/sendMarksReminders.js exports processReminders() - never imported anywhere.
```

reminderScheduler.js is genuinely good code. scheduleWeeklyQuizReminders already scans course_planner_data for topics matching 'Weekly Quiz' in the next seven days, resolves the batch's learners, computes an IST send time, deduplicates against scheduled_emails and inserts. scheduleIntermediateAssessmentReminders does the same for 'Intermediate Assessment' at D-7 for the trainer. This is precisely the planner-derived scheduling the orchestrator needs - it just needs a cron tick to call it.

B3 Critical state stored on an ephemeral filesystem

STATE	LOCATION	CONSEQUENCE
Mark-window extensions	uploads/mark-window-extensions.json	Lost on every deploy
Mark-entry open flags	uploads/mark-entry-open.json	Lost on every deploy
Generated planners (.xlsx/.csv/.json)	uploads/course-planner/	Lost on every deploy
Attendance / feedback uploads	backend/uploads/	Committed to git as a workaround

index.js:1105-1250 is explicit about the trade-off: "This avoids a DB dependency for a short-lived (approx. one day) value and works on the current deployment." On Render, container restarts are routine and not under the operator's control. When the file vanishes, every trainer's mark-entry window silently slams shut with no error and

no audit trail. Automation cannot depend on state that disappears.

B4 marks_entry_windows is never populated

The table is SELECTed in marksWindowService.js and UPDATED at index.js:1646. There is no INSERT anywhere in the codebase. getWindowStatus() therefore returns exists:false, is_open:false for any batch whose window row was not hand-created directly in Supabase. The formal window system is inert, and the real gate is the pair of ephemeral JSON files from B3.

GAP AN AUTOMATION OPPORTUNITY HIDING IN PLAIN SIGHT

Window rows are perfectly derivable: for each assessment-bearing date in course_planner_data, open at the session end time and close 24-48 hours later. Generating them at launch turns marks collection from a chase into a scheduled, self-reminding, self-escalating process.

B5 Authentication is absent, and the token is forgeable

```
// backend/index.js:4016 - /api/login
const token = Buffer.from(JSON.stringify(tokenPayload)).toString('base64');
//      ^ base64 of JSON. NOT signed. Anyone can mint {"role":"admin"}.

// backend/index.js:218 - authMiddleware
const decoded = jwt.verify(token, process.env.JWT_SECRET);
//      ^ expects a real signed JWT, so it REJECTS the token /api/login issues.

// Coverage: 1 of 153 routes uses authMiddleware (GET /api/attendance-report)
// Password check also accepts plain-text comparison when the hash is not bcrypt (line ~3995)
```

BLOCKER THIS MUST BE FIXED BEFORE A LAUNCH TRIGGER EXISTS

Today any anonymous caller can upload learners, schedule emails to every learner in any batch, approve extensions, or change marks. Adding POST /api/batch/launch to an unauthenticated surface hands that caller the ability to run the entire pipeline for any batch. Signed tokens plus server-side role checks are a prerequisite, not a follow-up.

B6 Live credentials committed to the repository

backend/.env.example contains a working SUPABASE_SERVICE_ROLE_KEY (which bypasses row-level security entirely), a real EMAIL_PASS, and a real JWT_SECRET. These are in git history and must be treated as compromised: rotate the Supabase service key, the mail credential and the JWT secret, then replace the file with placeholder values.

B7 No schema as code, no CI, no tests

- **migrations.sql is empty.** All 35 tables exist only inside the hosted Supabase project. There is no way to stand up a staging environment, and no review trail for schema changes.
- **.github contains no workflows.** The keep-alive workflow was deleted (commit e63570b) and nothing replaced it. No lint, no build check, no test run on push.
- **No automated tests.** For a system that emails learners and computes certification eligibility, a regression suite around the scorecard formulas and the scheduling offsets is the minimum.

B8 A second abandoned application

backend/src/ is a complete parallel implementation - server.js, api.js, mailer.js, emailSender.js, scheduler.js, templates.js, supabaseClient.js - written in CommonJS against an older schema. Nothing imports it. The root-level services/emailService.js is similarly orphaned. Beyond the maintenance confusion, src/scheduler.js is a live-looking dispatcher that a future maintainer could plausibly start.

B9 No workflow state model

Nothing in the database records that batch X has completed setup and is awaiting trainer assignment. Every action is an independent, fire-and-forget POST. There is therefore no answer to "what is pending for this batch?", no retry of a workflow (only of an individual email), no dependency ordering, and no audit of who triggered what. This is the single largest structural gap, and Section 5 is mostly about closing it.

B10 Fragile string matching and duplicated business rules

```
// index.js:4398  PDFT subject classification
if (raw.includes("digital")) return "digital";
if (raw.includes("cmos"))    return "cmos";

// index.js:4663  DVFT classification - substring match on a 2-letter token
if (name.includes("systemverilog") || name.includes("sv")) { sv += ... }
//                                     ^ matches ANY topic containing 'sv'

// Certification / placement thresholds are re-stated in the React footer text
// and do not agree with the backend rule (footer says project >= 60%, code uses 70%).
```

Topic names are free text typed into an Excel template. Any renaming silently drops marks from a subject bucket, and the learner's overall percentage changes with no error raised. This is the strongest candidate for an agentic solution (Section 7): classify once with judgement, then store the mapping and apply it deterministically.

5. Target Architecture - The One-Point Trigger

The design principle: a human declares intent once; the system derives every consequence from the course planner and drives it to completion, retrying and escalating on its own.

5.1 The trigger

```
POST /api/batch/launch      Authorization: Bearer <signed JWT>    role in (admin, manager)

{
  "batch_no":      "PDFT-2026-09",
  "domain":        "PD",
  "mode":          "Offline",           // Offline | Online
  "batch_type":    "PDFT",              // PDFT | DVFT | DFT
  "start_date":    "2026-09-01",
  "students":      24,
  "preferred_slot": "FN",
  "sessions":      { "s1": "09:30-11:00", "s2": "11:15-13:00", "s3": "14:00-17:00" },
  "trainers":      { "theory": "a@chippedge.com", "lab": "b@chippedge.com" },
  "learners":      { "source": "upload", "file_id": "..." },
  "options":       { "dry_run": true, "require_planner_review": true }
}

-> 202 Accepted
{ "run_id": "b7f3...", "stages": 11, "tasks_planned": 214, "console": "/batch-runs/b7f3" }
```

DESIGN DRY_RUN IS THE ADOPTION STRATEGY

With `dry_run: true` the orchestrator executes every stage that only reads, and instead of inserting into `scheduled_emails` it writes the would-be rows to a preview. Operators see exactly which 214 emails would go to whom and when, before trusting it. Run dry for two or three batches in parallel with the manual process, diff the output, then flip the flag.

5.2 New tables

```

create table batch_runs (
  id          uuid primary key default gen_random_uuid(),
  batch_no    text not null,
  charter     jsonb not null,          -- the launch payload, verbatim
  status      text not null,          -- planning|running|blocked|completed|cancelled
  current_stage text,
  dry_run     boolean not null default false,
  created_by  text not null,
  created_at  timestampz default now(),
  unique (batch_no, created_at)
);

create table batch_tasks (
  id          bigserial primary key,
  run_id      uuid references batch_runs(id) on delete cascade,
  batch_no    text not null,
  stage       text not null,          -- allocate_classroom | derive_schedule | ...
  task_key    text not null,          -- stable, human-readable identity
  depends_on  text[] default '{}',    -- task_keys that must succeed first
  run_after   timestampz not null,     -- earliest execution time
  status      text not null default 'pending',
  -- pending|ready|running|done|failed|needs_attention|skipped
  attempts   int not null default 0,
  max_attempts int not null default 5,
  payload     jsonb not null default '{}':jsonb,
  result      jsonb,
  error       text,
  idempotency_key text not null,
  updated_at  timestampz default now(),
  unique (run_id, task_key),
  unique (idempotency_key)
);

create index on batch_tasks (status, run_after);

create table batch_events (          -- append-only audit
  id          bigserial primary key,
  run_id      uuid, task_id bigint,
  kind        text not null,          -- stage_entered|task_done|escalated|agent_action
  actor       text not null,          -- user email | 'orchestrator' | 'agent:exception'
  detail      jsonb,
  created_at  timestampz default now()
);

create table automation_rules (      -- declarative, editable without deploy
  id          bigserial primary key,
  active      boolean default true,
  batch_type  text,                   -- null = all
  match_kind  text not null,          -- topic_ilike | stage | last_teaching_day
  match_value text,                   -- e.g. '%Weekly Quiz%'
  offset_days int not null default 0, -- negative = before the matched date
  send_time   text not null default '09:00', -- IST
  audience    text not null,          -- learners|trainer|coordinator|role:Management
  channel     text not null default 'email', -- email|push|both
  template_name text not null
);

```

5.3 The stage graph

```

LAUNCH
|
+-- validate_charter ..... batch_no unique, dates sane, trainers exist, learners parse
+-- allocate_classroom ..... reuses findAvailableClassroom(supabase, ...)
+-- check_licenses ..... reuses checkLicenseAvailability(supabase, domain, students)
+-- import_learners ..... reuses the /upload-learners dedupe logic
+-- generate_planner ..... spawns scripts/generate_course_planner.py (already exists)
+-- [GATE] planner_review ... optional human approval of the generated grid
+-- ingest_planner ..... xlsx -> csv -> course_planner_data, IN PROCESS (kills steps 4-6)
+-- assign_trainers ..... reuses assign-batch-trainers-by-module + conflict detection
|
+== derive_schedule <<< THE KEY STAGE >>>
|   reads every row of course_planner_data for this batch and, for each
|   automation_rule that matches, emits a dated task:
|
|       'Weekly Quiz'           -> learner reminder   D-0 12:00
|                               -> trainer prep       D-3 09:00
|       'Intermediate Assessment' -> trainer reminder D-7 09:00
|       'Final Assessment - *'   -> schedule mail + trainer + learner notices
|       'Mock Interview'         -> learner + trainer + coordinator mails
|       'Soft Skill*'            -> announcement
|       any assessment date      -> open marks window D+0 at session end
|                               -> close marks window D+1 23:59
|                               -> remind trainer     D+1 18:00 if rows missing
|                               -> escalate to coord. D+2 10:00 if still missing
|       max(date)                -> course closure + VPN disable notice D-3
|                               -> scorecard publish + review task      D+1
|
+-- schedule_welcome_emails .. reuses the /api/schedule-email template engine
+-- arm_recurring_jobs ..... attendance digest, weekly progress report
|
v
RUNNING --(reconciler tick, every minute)-->  CLOSING -->  CLOSED

```

STRENGTH ALMOST EVERY STAGE CALLS CODE THAT ALREADY EXISTS

This is a wiring exercise, not a rewrite. `findAvailableClassroom`, `checkLicenseAvailability`, the learner dedupe, the Python planner generator, the CSV ingest, the template scheduler and the two reminder schedulers are all written and working. The orchestrator supplies ordering, retries, state and an audit trail around them.

5.4 The reconciler

One cron tick replaces every manual button. It uses real PostgreSQL row locking through the pg Pool that backend/db.js already exposes, so multiple instances are safe.

```

cron.schedule("* * * * *", async () => {
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    const { rows: due } = await client.query(`
      SELECT * FROM batch_tasks
      WHERE status IN ('pending','ready','failed')
      AND run_after <= now()
      AND attempts < max_attempts
      AND NOT EXISTS (
        SELECT 1 FROM batch_tasks d
        WHERE d.run_id = batch_tasks.run_id
        AND d.task_key = ANY(batch_tasks.depends_on)
        AND d.status <> 'done')
      ORDER BY run_after
      FOR UPDATE SKIP LOCKED
      LIMIT 25`);

    for (const task of due) {
      await client.query("UPDATE batch_tasks SET status='running'," +
        " attempts=attempts+1 WHERE id=$1", [task.id]);

      try {
        const result = await HANDLERS[task.stage](task); // pure, idempotent
        await client.query("UPDATE batch_tasks SET status='done', result=$2" +
          " WHERE id=$1", [task.id, result]);
        await emitSuccessors(client, task, result);
      } catch (err) {
        const terminal = task.attempts + 1 >= task.max_attempts;
        const backoff = Math.min(2 ** task.attempts, 60); // minutes
        await client.query(
          "UPDATE batch_tasks SET status=$2, error=$3," +
          " run_after = now() + ($4 || ' minutes')::interval WHERE id=$1",
          [task.id, terminal ? 'needs_attention' : 'failed', err.message, backoff]);
        if (terminal) await escalate(task, err); // notify() + optional agent triage
      }
    }
    await client.query("COMMIT");
  } catch (e) { await client.query("ROLLBACK"); }
  finally { client.release(); }
});

```

5.5 Idempotency

Every handler must be safe to run twice, because retries and overlapping ticks are normal. The discipline is a deterministic idempotency_key per unit of work, with a unique index doing the enforcing rather than a read-then-write check.

```
idempotency_key = sha256([run_id, stage, batch_no, target, iso_date].join('|'))
```

examples

```

run:b7f3|derive_schedule|PDFT-2026-09|weekly_quiz|2026-09-12
run:b7f3|email|PDFT-2026-09|learner:asha@x.com|welcome|2026-09-01T09:00Z
run:b7f3|window_open|PDFT-2026-09|weekly|w3|2026-09-19

```

```

-- and on the delivery table itself, replacing the per-learner SELECT dedupe
-- currently done in a loop inside /api/schedule-email:
alter table scheduled_emails add column idempotency_key text;
create unique index on scheduled_emails (idempotency_key);
-- insert ... on conflict (idempotency_key) do nothing;

```

GAP THIS ALSO FIXES A PERFORMANCE PROBLEM

/api/schedule-email currently issues one SELECT per learner per template to check for duplicates - for 25 learners and 8 templates that is 200 round trips inside a request handler, with no transaction. A unique index plus a single batched insert ... on conflict do nothing replaces all of it.

5.6 Unified dispatch

Rather than each handler choosing email or push independently, one function fans a domain event out across channels according to automation_rules. Adding a channel later (WhatsApp, Slack, in-app inbox) becomes a change in one place.

```
async function dispatch(event) {
  // event = { kind, batch_no, when, audience, template_name, vars }
  const rules = await matchingRules(event);
  for (const rule of rules) {
    const to = await resolveAudience(rule.audience, event.batch_no);
    if (rule.channel !== 'push')
      await queueEmail({ to, template: rule.template_name, at: event.when,
        vars: event.vars, idempotency_key: keyOf(event, rule) });
    if (rule.channel !== 'email')
      notify(to, { title: render(rule, 'title'), body: render(rule, 'body'),
        data: { type: event.kind, batch_no: event.batch_no } });
  }
  await appendEvent(event);      // batch_events - full audit
}
```

6. Option A - Deterministic Orchestrator (the spine)

This alone delivers the requested outcome: one trigger, everything downstream automatic. It uses no AI. It is predictable, cheap, testable and auditable - which is what you want for anything that emails learners or gates certification.

6.1 What it automates completely

OUTCOME	DERIVED FROM	REPLACES
Classroom + license booking	charter students/slot/date range	Step 1
Planner generation and ingest	charter domain/type/start_date	Steps 2, 4, 5, 6
Learner import	charter learners source	Step 7
Welcome / joining email series	email_templates offset_days + send_time	Step 9
Weekly quiz reminders	planner rows matching Weekly Quiz	already written, never called
Intermediate assessment prep	planner rows matching Intermediate	already written, never called
Mark window open / close	assessment dates + session end_time	Step 10, plus B3 and B4
Missing-marks chase + escalation	scores tables vs planner obligations	manual follow-up
Mock interview comms	planner mock interview row	Step 12
Soft skills announcement	planner soft skill row	Step 13
Final assessment comms	planner final assessment rows	Step 14
Course closure + VPN notice	max(date) for the batch	Step 15
Scorecard publish + review task	closure + 1 day	manual
Failure recovery	reconciler backoff + escalate	manual re-click

6.2 Rules become data, not code

Every timing decision currently buried in a handler moves into automation_rules, so a coordinator can change 'remind the trainer 7 days before' to 5 days without a deployment.

MATCH_KIND	MATCH_VALUE	OFFSET	TIME	AUDIENCE	TEMPLATE
topic_iline	%Weekly Quiz%	0	12:00	learners	Weekly Quiz Reminder
topic_iline	%Weekly Quiz%	-3	09:00	trainer	Trainer Quiz Prep
topic_iline	Intermediate Assessment%	-7	09:00	trainer	Trainer Inter. Prep
topic_iline	%Mock Interview%	-2	10:00	learners	Mock Interview Notice
topic_iline	Final Assessment%	-5	09:00	learners	Final Assessment Notice
last_teaching_day	(n/a)	-3	09:00	role:Management	Course Closure / VPN
stage	scorecard_ready	+1	10:00	coordinator	Scorecard Review

6.3 The operator console

One new React page, /batch-runs, closes the visibility gap (B9). It is what makes the automation trustworthy rather than opaque.

- **Run timeline.** Every stage with status, duration and the actor that completed it.

- **Task list.** Filter by needs_attention. Per-task retry, skip and edit-payload actions.
- **Scheduled preview.** The full forward calendar of queued emails and pushes for the batch.
- **Dry-run diff.** Side-by-side of what a re-derive would add, change or remove - essential after a topic is rescheduled.
- **Audit.** batch_events rendered as a feed, including any agent action and its reasoning.

6.4 Handling reschedules

There is already an /api/update-actual-date endpoint and a date_change_audit table, but changing a date today does not revise emails that were already queued. Under the orchestrator, a date change raises a re-derive task for that batch which recomputes the obligation set and reconciles the difference: cancel superseded scheduled_emails rows that are still in 'scheduled', insert the new ones, and leave anything already 'sent' alone with an event recorded.

7. Option B - The Claude Code Agentic Layer

An agent earns its place where the input is messy, the rule is hard to write down, or a human would otherwise read context and decide. It has no place in date arithmetic or eligibility maths. The boundary below is the most important part of this section.

7.1 Where an agent must NOT be used

BLOCKER KEEP THE LLM OFF THE CRITICAL PATH

Never delegate: computing send dates, deciding which learners receive an email, marks arithmetic, scorecard weightage, certification or placement eligibility, attendance percentage, or anything that writes directly to `scheduled_emails`. These are deterministic rules with legal and financial weight, they must be unit-testable, and they must produce identical output on every run. Section 6 covers all of them.

7.2 Five agents that do earn their place

A1 Planner Ingest and Classification Agent

The problem is B10. Topic names are free text typed into six different Excel templates, and classification is done with substring matching so fragile that a topic containing the letters 'sv' lands in the SystemVerilog bucket. Renaming a topic silently changes a learner's overall percentage.

- **Input:** the newly generated planner rows for a batch.
- **Job:** map each row to a canonical taxonomy (subject, assessment kind, weightage group), and flag anything ambiguous or unmapped rather than guessing.
- **Output:** a proposed mapping, presented for one-click human confirmation.
- **Key property:** the confirmed mapping is STORED in a `topic_taxonomy` table. From then on classification is a deterministic lookup. The agent runs once per planner, not once per calculation.

A2 Exception Triage Agent

Fires only when the reconciler marks a task needs_attention. It reads the run context and either applies a whitelisted remediation or drafts an escalation with a recommendation.

EXCEPTION	AGENT ACTION	AUTONOMY
No classroom free for the range	Propose alternate slots or a split; show the occupancy conflict	Recommend only
Licenses short by N	Identify batches releasing licenses before start_date	Recommend only
Trainer unavailable for a topic	Query available-trainers-by-schedule, propose a swap	Auto with approval
Planner row missing trainer_email	Infer from batch_module_trainers for that module	Auto
Email permanently_failed	Classify: bad address / Mailjet block / transient. Re-queue or flag	Auto for transient
Marks missing past window close	Draft the chase to the right trainer with the exact gaps	Auto with approval
Duplicate learner rows on import	Diff and propose the canonical record	Recommend only

A3 Communications Drafting Agent

Generates and refreshes email_templates bodies, personalises closure and scorecard narratives, and adapts tone per audience. Output always lands in a review queue with a diff against the current template - it never reaches a learner without a human approving the text once.

A4 Data Quality Auditor (scheduled, read-only)

A nightly agent that reads the system the way an experienced coordinator would and produces a digest plus batch_tasks for anything actionable.

- Learners with missing or malformed email addresses, or on a batch with no planner
- Planner dates with no trainer assigned, or a trainer double-booked across batches
- Attendance trending below the 85% certification threshold, early enough to intervene
- Assessments whose marks are absent after the window closed
- Scorecard overrides that diverge sharply from the computed value, or lack substantive remarks
- scheduled_emails stuck in processing, or permanently_failed clusters suggesting a provider issue
- Batches approaching their end date with no closure task queued

A5 Build-Time Agent - Claude Code itself

Distinct from the runtime agents: this is Claude Code performing the migration described in Section 8. It is well suited to the mechanical, wide-surface work that dominates Phase 0 and Phase 1.

- Generate migrations.sql by introspecting the live Supabase schema (closes B7)
- Add signed-JWT auth plus server-side role checks across ~110 mutating routes (closes B5)
- Move the two JSON file stores into Postgres and rewrite their read/write paths (closes B3)
- Delete backend/src/, email-worker.js, courseApplicationScheduler.js and services/ after proving they are unreferenced (closes B1, B8)
- Split index.js (10,763 lines) into domain routers without behaviour change
- Write the regression suite around the scorecard formulas and scheduling offsets

7.3 How the runtime agents are hosted

```

Render Cron Job or a small always-on worker
  |
  v
Claude Agent SDK (@anthropic-ai/claude-agent-sdk)    model: claude-sonnet-5
or headless: claude -p "<task>" --output-format json  escalate: claude-opus-5
  |
  v
MCP server (a thin wrapper over the EXISTING REST API - no new business logic)
  |
  +-- read tools (unrestricted)
  |   get_batch_run, list_tasks, get_planner, get_scores, get_attendance,
  |   find_available_trainer, get_classroom_occupancy, get_mail_status
  |
  +-- write tools (allowlisted, audited, reversible)
  |   create_task, propose_mapping, requeue_email, raise_escalation,
  |   propose_trainer_swap, draft_template
  |
NOT exposed as tools: direct SQL, insert into scheduled_emails,
                      update to any *_scores table, delete anything

```

7.4 Guardrails

- **Separation of powers.** The agent may create batch_tasks; only the deterministic reconciler executes them. An agent can never send an email directly.
- **Allowlisted writes.** Every write tool is enumerated, schema-validated, and reversible. There is no generic SQL tool.
- **Approval gates.** Anything that reaches a learner, changes a mark, or moves a date requires human approval recorded in the approvals table.
- **Full auditability.** Every agent action appends to batch_events with actor = 'agent:<name>', the tool call, and the stated reasoning. Reviewable after the fact.
- **Budget ceilings.** Per-run token and cost caps; the agent degrades to raising a plain escalation when exhausted, never to guessing.
- **Bake period.** Every agent ships read-only first. It writes its recommendation to the console for two to four weeks; only after the recommendations are consistently right is the write tool enabled.

- **Prompt-injection posture.** Planner topic names, learner names and uploaded files are untrusted input. Agents treat them as data, never as instructions, and write tools validate arguments against the schema independently of what the model asked for.
-

8. Recommendation and Roadmap

8.1 Recommended shape - hybrid, deterministic spine first

DESIGN BUILD OPTION A COMPLETELY. ADD OPTION B AT THE EDGES.

The deterministic orchestrator carries 100% of the happy path and is what actually delivers the one-trigger outcome. Agents then attach at exactly four points: planner classification on ingest, exception triage on needs_attention, template drafting into a review queue, and a nightly read-only audit. If every agent were switched off tomorrow, batches would still run end to end - they would just escalate more to humans. That is the correct failure mode.

8.2 Phased plan

Phase 0 - Stabilise (1-2 days, blocking)

- Rotate the Supabase service-role key, mail password and JWT secret; replace .env.example with placeholders (B6)
- Choose the index.js cron as the single dispatcher; quarantine email-worker.js, backend/src/, courseApplicationScheduler.js and services/ (B1, B8)
- Move mark-window-extensions.json and mark-entry-open.json into Postgres tables (B3)
- Introspect the live schema into migrations.sql (B7)

Phase 1 - Foundation (3-5 days)

- Real signed JWTs; authMiddleware on every mutating route; server-side role checks replacing the client-side menu gate (B5)
- Create batch_runs, batch_tasks, batch_events, automation_rules, approvals
- Implement the reconciler tick with FOR UPDATE SKIP LOCKED and exponential backoff
- Add idempotency_key plus a unique index to scheduled_emails and replace the per-learner dedupe loop

Phase 2 - The single trigger (1 week)

- POST /api/batch/launch with charter validation and dry_run
- Stage handlers wrapping the existing functions - no rewrites
- In-process planner generate -> ingest, eliminating the download/upload round trip (steps 4-6)
- derive_schedule reading course_planner_data
- Wire the four dead schedulers plus sendMarksReminders as task handlers (B2)
- Generate marks_entry_windows rows from assessment dates (B4)

Phase 3 - Rules, notifications, console (1 week)

- Move every hardcoded offset into automation_rules
- Unified dispatch() across email, push and in-app
- The /batch-runs operator console with timeline, retry, approve and dry-run diff
- Re-derive on date change, reconciling already-queued mail

Phase 4 - Agentic layer (1-2 weeks)

- MCP wrapper over the existing API with read tools first
- Data Quality Auditor nightly, read-only, output to the console
- Planner Ingest agent with human confirmation, writing to topic_taxonomy
- Exception Triage read-only, then promoted to auto for the two safest classes after a bake period

8.3 Effort and sequencing

PHASE	EFFORT	SHIPS INDEPENDENTLY?	VALUE ON ITS OWN
0 - Stabilise	1-2 days	Yes	Removes duplicate-send and data-loss risk
1 - Foundation	3-5 days	Yes	Closes the security hole; adds state
2 - Trigger	1 week	Yes	The headline outcome: one-click launch
3 - Rules + console	1 week	Yes	Ops self-service and visibility
4 - Agents	1-2 weeks	Yes	Fewer escalations reaching a human

Total: approximately four to six weeks for one experienced engineer, assuming Phases 0 and 1 are not skipped. They should not be - Phase 2 without Phase 1 means publishing an unauthenticated endpoint that runs an entire batch pipeline for any caller.

8.4 What a human still does afterwards

- **Launch a batch.** One form, one trigger.
- **Review the generated planner.** An optional gate, recommended for the first months.
- **Enter marks and attendance.** This is the actual teaching work and should stay human.
- **Review the scorecard.** Including the override and remarks flow.
- **Approve exceptions.** Only what the system could not resolve on its own.

Everything else - every email, every reminder, every push, every window open and close, every escalation, closure and report - becomes a derived consequence of the launch and the planner.

8.5 Risks

RISK	MITIGATION
Automated mail storm to learners	dry_run default on; per-batch send ceiling; console preview; kill switch that pauses the reconciler without stopping the app
Wrong dates from a mis-parsed planner	planner_review gate; the Ingest agent flags ambiguity rather than guessing; dry-run diff before every re-derive
Reconciler double-executes	FOR UPDATE SKIP LOCKED plus unique idempotency_key; both are belt and braces
Agent takes a wrong action	Read-only bake period; allowlisted reversible writes; approval gates; full batch_events audit
Render restarts mid-run	All state in Postgres after Phase 0; tasks resume on the next tick by design
Regression in scorecard maths	Phase 1 test suite pinning the current formulas before anything else is touched

9. Appendix - Evidence Index

Every claim in this document is anchored to a file and, where useful, a line number, so each finding can be verified independently.

FINDING	LOCATION
Per-minute email dispatcher	backend/index.js:9815-9927
Keep-alive ping	backend/index.js:9931
Schedulers imported, never called	backend/index.js:16-22 (verified by repo-wide grep)
Weekly quiz reminder logic	backend/reminderScheduler.js:62-144
Intermediate reminder logic	backend/reminderScheduler.js:147-219
Marks reminder worker, never imported	backend/workers/sendMarksReminders.js:54
Second dispatcher	backend/email-worker.js:20-135 (npm run worker)
Third dispatcher (dead)	backend/src/scheduler.js - polls status='pending'
Fourth dispatcher (dead)	backend/courseApplicationScheduler.js:7 - while(true)
Window extension file store	backend/index.js:1105-1174
Mark-entry-open file store	backend/index.js:1183-1249
marks_entry_windows never inserted	backend/marksWindowService.js:9 (SELECT), index.js:1646 (UPDATE)
Unsigned login token	backend/index.js:4016
authMiddleware expects a real JWT	backend/index.js:218-238
Auth coverage: 1 of 153 routes	backend/index.js:10722 is the only guarded route
Plain-text password fallback	backend/index.js:3990-3996
Live secrets committed	backend/.env.example
Empty migrations file	backend/migrations.sql (0 bytes)
Planner generate / convert / download	backend/routes/coursePlanner.js:101, 218, 300
Planner CSV re-upload	backend/index.js:1839 (POST /upload-course-planner)
Template offset scheduling	backend/index.js:7091-7143 (offset_days + send_time)
PDFT subject classification	backend/index.js:4398-4405
DVFT substring match on 'sv'	backend/index.js:4663
Scorecard weightage and rules	backend/index.js:4495-4528
Push fan-out helper	backend/push/sendPush.js:114-178
Mailjet sender	backend/emailSender.js:16-130
Role menus (client-side gate only)	frontend/src/App.js:40-120
Manual hub	frontend/src/pages/HomeDashboard.js:282, 333, 373

Scope note

This analysis was read-only. No application file was modified in producing this report. The findings reflect branch rollback-stable at commit 29eab18. Line numbers in the appendix refer to that commit and will drift as the code changes.

